



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez.

Asignatura: Programación orientada a objetos.

Grupo: 07

No de Práctica(s): Proyecto - Etapa 1

Integrante(s): 323331229

426060019

323217156

312109239

323195386

321321976

*No. de lista o
brigada:* 6

Semestre: 3^{ro}

Fecha de entrega: 28/09/2026

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción	3
1.1. Planteamiento del problema	3
1.2. Motivación	3
1.3. Objetivos	3
2. Marco Teórico	5
3. Desarrollo	6
4. Resultados	14
5. Conclusiones	17

1. Introducción

1.1. Planteamiento del problema

El proyecto consiste en crear la simulación de un sistema hospitalario para gestionar las acciones del personal médico y de los pacientes. Para esta primera etapa el sistema se compone de cuatro clases fundamentales:

Sistema: Cuyos atributos son `nombreHospital`, `noMedicos`, `noEnfermeros`, `noPacientes`. Y de comportamientos para esta etapa son `registroMedico`, `registroEnfermero`, `registroPaciente`, `asignarPaciente` (es un método sobrecargado).

Médico: Cuyos atributos son `nombre`, `cedula`, `especialidad`, `noPacientes`. Y de comportamientos para esta etapa son `registroEnSistema`, `solicitarPaciente`, `darConsulta`, `darTratamiento`, `verListaPacientes`.

Enfermero: Cuyos atributos son `nombre`, `cedula`, `especialidad`, `noPacientes`. Y de comportamientos para esta etapa son `registroEnSistema`, `verListaPacientes`, `darTratamiento`.

Por último la clase **Paciente:** Para el registro y mecanismos para determinar el estado de atención de cada paciente en consulta o en tratamiento.

Considerando las siguientes restricciones:

Los pacientes se asignan con base a la especialidad que requiere de atención.

Los médicos solo pueden dar consulta a un paciente a la vez, pero cada médico solo puede tener hasta 10 pacientes de la especialidad.

Los enfermeros pueden cuidar hasta 3 pacientes a la vez de la misma especialidad. Se asume que debe existir alguna estructura lineal para llevar el registro de pacientes y personal médico.

El método `verListaPacientes` debe ordenar por alfabeto descendente, mostrando apellidos y nombres además de la especialidad.

En esta etapa debe existir un mecanismo para determinar si un paciente está en consulta o en tratamiento.

1.2. Motivación

El objetivo de este proyecto es poner en práctica los conceptos de programación orientada a objetos vistos durante el parcial, como el diseño de clases, la instanciación de objetos y la interacción entre éstos. Se intenta abstraer el funcionamiento de un hospital real en un modelo computacional estructurado.

1.3. Objetivos

- **General:** Comprender y aplicar los conceptos de la Programación Orientada a Objetos en la resolución de un problema cercano a la realidad, como lo es la gestión de un sistema hospitalario.
- **Específicos:**
 - Diseñar e instanciar las cuatro clases principales del modelo: Sistema, Médico, Enfermero y Paciente, definiendo sus respectivos atributos y comportamientos.

- Implementar la sobrecarga de métodos para la asignación de pacientes (**asignarPaciente**) dentro de la clase Sistema.
- Administrar el registro del personal médico y de los pacientes ingresados.
- Validar las restricciones del hospital, asegurando que la asignación sea por especialidad, respetando el límite de un paciente a la vez por médico (máximo 10 de la especialidad) y hasta tres pacientes simultáneos por enfermero.
- Desarrollar un mecanismo de control que permita determinar en todo momento si un paciente se encuentra en consulta o en tratamiento.
- Aplicar algoritmos de ordenamiento en el método **verListaPacientes** para desplegar la información (apellidos, nombres y especialidad) en orden alfabético descendente.

2. Marco Teórico

1. Programación orientada a objetos: Se basa en la creación de objetos y la interacción con éstos, para el diseño y desarrollo de aplicaciones de computadora en la que los datos y los procedimientos para la manipulación de éstos están estrechamente ligados. [1]
2. Clase: Son plantillas para la creación de objetos. Define las variables y los métodos que son comunes para todos los objetos de un cierto tipo.[4]
3. Objeto: Representan cosas del mundo real, los elementos de un objeto se dividen en dos categorías principales atributos y métodos. [5]
4. Atributos: Los atributos, incluyen características sobre el objeto, por ejemplo, si consideramos un objeto Coche algunas de sus propiedades serán: el color, la marca, el modelo o el año de fabricación. [5]
5. Métodos: Los métodos definen las acciones que se pueden realizar con el objeto, por ejemplo, para el objeto Coche, los métodos podrían ser acelerar, frenar o girar. [5]
6. Instanciar: Una vez definida la clase se pueden crear objetos a partir de ésta, a este proceso se le conoce como crear instancias de una clase o instanciar una clase, cada uno de los objetos o instancias tiene su propia copia de las variables definidas en la clase de la cual son instanciados y comparten la misma implementación de los métodos. [3]
7. Sobrecarga de métodos (overload): La sobrecarga de métodos es la creación de varios métodos con el mismo nombre, pero con diferente lista de tipos de parámetros. Java utiliza el número y tipo de parámetros para seleccionar cuál definición de método ejecutar. [6]
8. Encapsulamiento: Sucede cuando un elemento del objeto (atributo o método) es envuelto en una capa protectora implica que los datos del objeto están protegidos o ocultos dentro de él , el programa no puede acceder de manera directa a los datos encapsulados de un objeto, esto se tiene que realizar con ayuda de los métodos del objeto. [2]
9. Constructor: Es un método que tiene el mismo nombre que la clase y cuyo propósito es inicializar los atributos de un nuevo objeto se ejecuta automáticamente cuando se crea un objeto o instancia de la clase. [3]
10. Mensajes: Los objetos entran en interacción enviándose mensajes entre ellos. Una vez que el objeto reciba un mensaje, actuará. La acción puede ser el envío de otros mensajes, el cambio de su estado, o la ejecución de toda otra tarea que el objeto deba realizar. Los objetos de un programa. interactúan y se comunican entre sí por medio de mensajes. [3]
11. Estructura de datos lineal: Es aquella en la que los elementos se organizan de forma secuencial, es decir, uno detrás de otro. En este tipo de estructuras, cada elemento tiene un único elemento anterior (predecesor) y un único elemento posterior (sucesor), con excepción del primero y el último. [7]

3. Desarrollo

Empezamos diseñando las cuatro clases principales.

- **Clase Sistema:** Centraliza la gestión del hospital. Incluye atributos como `nombreHospital` y contadores de personal/pacientes, además de manejar los registros y la asignación de pacientes.
 1. Se declaran atributos privados para el `nombreHospital` y contadores enteros `noMedicos`, `noEnfermeros`, `noPacientes` para llevar un control numérico exacto de los registros.
 2. Para cumplir con el requerimiento de implementar estructuras lineales, se declaran tres listas utilizando la interfaz `List` y su implementación `ArrayList`. Estas estructuras dinámicas permiten almacenar y administrar colecciones de objetos `Medico`, `Enfermero` y `Paciente` sin un tamaño fijo predefinido.
- **Constructor:** El constructor `Sistema(String nombreHospital)` inicializa el estado del objeto. Asigna el nombre, establece los contadores en 0, y lo más importante, reserva el espacio en memoria instanciando los tres `ArrayList`. Sin este paso las listas apuntarían a nulo y causarían errores de ejecución al intentar registrar datos.
- **Getters:** Proveen acceso de solo lectura a los atributos privados permitiendo que otras clases consulten la cantidad de pacientes o médicos, pero evitando que modifiquen los contadores directamente.
- **Métodos de Registro:** Los métodos `registroMedico`, `registroEnfermero` y `registroPaciente` cumplen una doble función:
 1. Utilizan el método `.add()` de las listas para insertar el nuevo objeto recibido por parámetro.
 2. Inmediatamente después incrementan su respectivo contador en 1 manteniendo sincronizada las estadísticas del hospital.
- **Sobrecarga de Métodos:** Se implementa la sobrecarga en el método `asignarPaciente`, definiéndolo dos veces con el mismo nombre pero con diferentes firmas (una recibe un objeto `Medico` y la otra un objeto `Enfermero`).
 1. **Validación:** Dentro de estos métodos se aplican las restricciones del hospital mediante una estructura condicional `if`.
 2. Primero, se extrae la especialidad del paciente y se compara con la del personal usando el método `.equals()` para garantizar que coincidan.
 3. Mediante el operador lógico `&&` se verifica que el médico tenga menos de 10 pacientes asignados (`< 10`), o en el caso del enfermero, menos de 3 pacientes (`< 3`).
 4. Solo si ambas condiciones se cumplen simultáneamente, el paciente es asignado; de lo contrario, el sistema rechaza la operación y notifica el error en consola, asegurando que no se exceda la capacidad del personal.

- **Clase Médico:** Representa al personal médico del hospital, encapsulando su información profesional y la lógica para atender a sus pacientes.
 1. Se declararon los atributos básicos solicitados: `nombre`, `cedula`, `especialidad` y el contador `noPacientes`.
 2. Para la restricción de que los médicos solo pueden dar consulta a un paciente a la vez se implementó el atributo `pacienteSiendoConsultado` que actúa como una variable de bloqueo que indica si el doctor está libre u ocupado.
 3. Se utilizó una estructura lineal (`List<Paciente> listaPacientes`) para llevar el registro individual de los pacientes asignados a cada médico.
- **Constructor:** El método `Medico(String nombre, String cedula, String especialidad)` se encarga de inicializar el estado del objeto al momento de su creación.
 1. Asigna los valores recibidos por parámetro a los atributos correspondientes (`nombre`, `cedula` y `especialidad`) haciendo uso de la palabra reservada `this`.
 2. Establece el contador inicial de pacientes (`noPacientes`) en cero (`this.noPacientes = 0;`).
 3. Reserva la memoria dinámica instanciando la estructura lineal `this.listaPacientes = new ArrayList<>();` paso fundamental para evitar errores de referencia nula al momento de registrar al primer paciente.
 4. Inicializa la variable de control lógica `pacienteSiendoConsultado` en `null`, estableciendo por defecto que el médico recién registrado en el sistema se encuentra libre y disponible para iniciar consultas.
- **Getters:** `getNombre()`, `getEspecialidad()`, `getNoPacientes()` y `getCedula()` solo devuelven la información, sin permitir que se modifique desde afuera.
- **Método registroEnSistema:** Igual que en `Paciente`, esta clase se apoya en el `Sistema` para registrarse a sí misma llamando a `sistema.registroMedico(this)`.
- **Método agregarPaciente:** Antes de añadir un objeto a la lista evalúa la condición `if (this.noPacientes < 10)`. Esta validación garantiza que cada médico cumpla con la restricción de tener un límite máximo de 10 pacientes asignados a su cargo.
- **Mecanismos de Consulta y Tratamiento:** Los métodos `darConsulta` y `darTratamiento` implementan las reglas de negocio para determinar si un paciente está en consulta o recibiendo tratamiento médico.
 1. **darConsulta:** Primero comprueba que el médico esté libre luego verifica mediante `contains()` que el paciente le pertenezca y que no esté actualmente ocupado. De ser exitoso, cambia el estado del paciente (`setEnConsulta(true)`) y bloquea al médico asignándolo a la variable `pacienteSiendoConsultado`.
 2. **darTratamiento:** Si el paciente está en la lista y aún no recibe tratamiento, actualiza las banderas del paciente (`setEnConsulta(false)` y `setEnTratamiento(true)`). Al concluir este paso libera al médico restableciendo su variable de control a `null`, permitiéndole atender a la siguiente persona en la lista.

- **Método `verListaPacientes`:** Para satisfacer la restricción de visualización, se hace uso de las bibliotecas de Java invocando `listaPacientes.sort(Comparator.comparing(Paciente::getNombre))`. Esta instrucción ordena los elementos de la estructura lineal y garantiza que el registro se imprima en pantalla en orden alfabético descendente, mostrando el nombre y la especialidad de cada paciente.
- **Clase `Enfermero`:** Modela al personal de enfermería gestionando sus datos profesionales y su capacidad para administrar tratamientos a los pacientes asignados.
 1. Se declaran de tipo `private` los atributos solicitados: `nombre`, `cedula`, `especialidad` y el contador `noPacientes`.
 2. Se emplea una estructura lineal dinámica declarada como `List<Paciente> listaPacientes` para registrar individualmente a los pacientes que están bajo el cuidado del enfermero.
- **Constructor `Enfermero(String nombre, String cedula, String especialidad)`:** Es el método especial encargado de inicializar el estado del objeto al instanciarlo.
 1. Asigna los valores recibidos por parámetro a los atributos internos usando la palabra reservada `this` para evitar ambigüedades.
 2. Establece el contador inicial de pacientes en cero (`this.noPacientes = 0;`).
 3. Reserva el espacio en la memoria dinámica para la estructura lineal (`this.listaPacientes = new ArrayList<>();`). Este paso es crítico para instanciar la lista y evitar que el programa lance un error de nulo al intentar agregar al primer paciente.
- **Getters:** Se implementaron los métodos públicos (`getNombre`, `getEspecialidad`, `getNoPacientes` y `getCedula`) para exponer el estado del enfermero de forma segura y en modo de solo lectura.
 1. **`agregarPaciente`:** Aplica de forma estricta la restricción operativa del personal de enfermería mediante la estructura condicional `if (this.noPacientes < 3)`, el algoritmo garantiza que un enfermero solo pueda cuidar hasta un máximo de 3 pacientes a la vez.
 2. **`darTratamiento`:** Controla el flujo clínico y determina si el paciente está habilitado para recibir cuidados, realiza validaciones comprobando que la lista no esté vacía y que el paciente buscado pertenezca a dicho enfermero (usando el método `.contains()`), verifica explícitamente que la persona ya haya salido de su consulta médica (`!paciente.estaEnConsulta()`). Si cumple las condiciones, actualiza la bandera lógica del paciente invocando `setEnTratamiento(true)` y notifica el estado en consola.
 1. **`registroEnSistema`:** Facilita la comunicación entre objetos al enviar la propia instancia del enfermero (mediante `this`) hacia el método de registro en la clase `Sistema`.

2. **verListaPacientes:** Cumple con el requisito de visualización aplicando un algoritmo de ordenamiento a la estructura lineal mediante `listaPacientes.sort(Comparator.comparing)`. Esto garantiza que la información se imprima en pantalla en orden alfabético descendente, detallando el nombre y la especialidad de cada paciente a su cargo.
- **Clase Paciente:** Almacena el nombre y la `especialidadAtencion` requerida. Representa a un paciente dentro del sistema hospitalario. Guarda su información básica y su estado actual (si está en consulta o en tratamiento).
 - **Atributos y Mecanismo de Estado:**
 1. Se declaran de forma privada los atributos base definidos en las especificaciones: `nombre` y `especialidadAtencion`.
 2. Para cumplir con el requerimiento de implementar un mecanismo que determine si un paciente está en consulta o en tratamiento, se declararon dos banderas booleanas encapsuladas: `enConsulta` y `enTratamiento`. Estas variables actúan como el semáforo lógico del estado del paciente.
 - **Constructor Paciente(String nombre, String especialidadAtencion):**
 1. Inicializa los atributos del objeto asignando el nombre y la especialidad requerida mediante la palabra reservada `this`.
 2. Por defecto, inicializa las banderas booleanas `enConsulta` y `enTratamiento` en `false`. Esto asume lógicamente que un paciente recién creado en el sistema aún no está siendo atendido ni medicado.
 - **Getters y Setters:**
 1. **Getters:** Los métodos `getNombre` y `getEspecialidadAtencion` permiten que la clase `Sistema` consulte qué tipo de médico o enfermero necesita el paciente para validarlo en el método de asignación. `estaEnConsulta` y `estaEnTratamiento` retornan el valor de las banderas booleanas.
 2. **Setters:** Los métodos `setEnConsulta` y `setEnTratamiento` permiten que los objetos de tipo `Medico` y `Enfermero` modifiquen el estado clínico del paciente (cambiando los booleanos a `true` o `false`) durante el flujo de trabajo sin romper el encapsulamiento de la clase.
 - **Comportamientos Específicos:**
 1. **registroEnSistema:** Facilita la integración enviando la propia instancia del paciente (`this`) como argumento al método de registro general en la clase `Sistema`.
 2. **solicitarConsulta y verTratamiento:** Implementan los comportamientos explícitamente solicitados para esta clase. El método `verTratamiento` evalúa internamente la bandera booleana `enTratamiento` e imprime un mensaje en consola únicamente si el valor es verdadero, confirmando así su estado de manera dinámica.

3. **verTratamiento y verConsulta:** Cumplen con el requisito de determinar el estado clínico actual del usuario. Ambos métodos evalúan internamente sus respectivas banderas booleanas (**enTratamiento** y **enConsulta**) e imprimen un mensaje confirmatorio en consola únicamente si el valor es verdadero.
- **Clase Main:** Es el entorno de pruebas donde se instancian los objetos y se ejecutan secuencialmente todos los métodos para validar los requerimientos de la Etapa 1.
 - **Importación de Bibliotecas:** Se importa `java.util.Scanner` para habilitar la lectura de datos ingresados por el usuario desde la terminal.
 1. Se declara la variable local `nombreHospital` y se emplea un ciclo `do-while` para obligar al usuario a ingresar un dato válido.
 2. Mediante la condición `nombreHospital.isEmpty()`, el programa atrapa los ingresos vacíos como presionar la tecla Enter sin escribir texto, mostrando un mensaje de error y repitiendo la solicitud para evitar inicializar el sistema con un nombre nulo.
 - **Instanciación de Objetos:** Se reserva la memoria y se construyen los objetos necesarios para la simulación:
 1. Una instancia central de la clase **Sistema**.
 2. Dos instancias de **Medico**, dos de **Enfermero** y dos de **Paciente**, todas inicializadas utilizando explícitamente `Cardiología` como especialidad para forzar las coincidencias lógicas en las pruebas de asignación posteriores.
 - **Pruebas de Registro y Consulta de Datos:**
 1. **Registro Dual:** Comprueba la correcta comunicación entre clases al registrar objetos de dos formas distintas: a través de los métodos propios de la instancia. `medico1.registroEnSistema(hospital)` y directamente a través del controlador central `hospital.registroMedico(medico2)`.
 2. Imprime en consola los contadores (`getNoMedicos`, etc.) y devuelve las estructuras lineales completas del sistema para verificar que todos los registros se guardaron exitosamente.
 3. **Iteración For-Each:** Emplea un ciclo `for` (`Paciente paciente : hospital.getPacientes()`) para recorrer dinámicamente la lista de pacientes del hospital, ejecutando `solicitarConsulta()` y extrayendo el nombre y especialidad de cada uno en cada iteración.
 - **Prueba de Sobrecarga y Asignación:** Ejecuta el método sobrecargado `asignarPaciente` enviando tanto a médicos como a enfermeros por parámetro, esto detona las validaciones internas de la clase **Sistema** para confirmar que la especialidad coincida y que no se superen los límites permitidos (10 pacientes para médicos, 3 para enfermeros).
 - Simula el ciclo de la atención médica evaluando las banderas booleanas de los pacientes.
 1. Imprime el estado inactivo inicial del paciente invocando `verConsulta()` y `verTratamiento()`.

2. Ejecuta `medico1.darConsulta(paciente1)`, detonando el cambio de estado a ocupado.
3. Valida la transición a `darTratamiento()` (probado tanto con el médico como con el enfermero), comprobando que el algoritmo verifique si el paciente ya terminó su consulta previa antes de proceder con el tratamiento.

■ Ordenamiento y Cierre:

1. Invoca `verListaPacientes()` desde los objetos del personal, lo que corrobora visualmente en la terminal que la estructura lineal de pacientes ha sido manipulada y ordenada correctamente en formato alfabético descendente.
2. Ejecuta `scanner.close()` para liberar de manera segura el flujo de memoria asociado a la entrada de la consola.

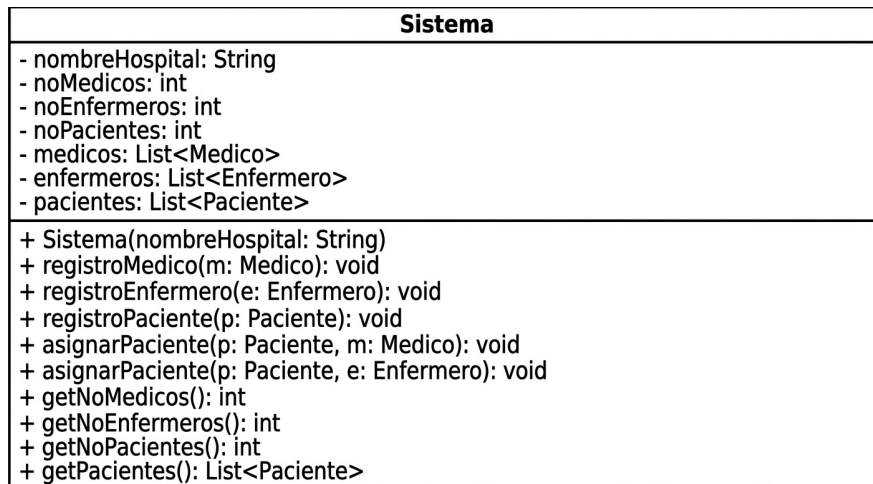


Figura 1: Diagrama de clase UML para Sistema.

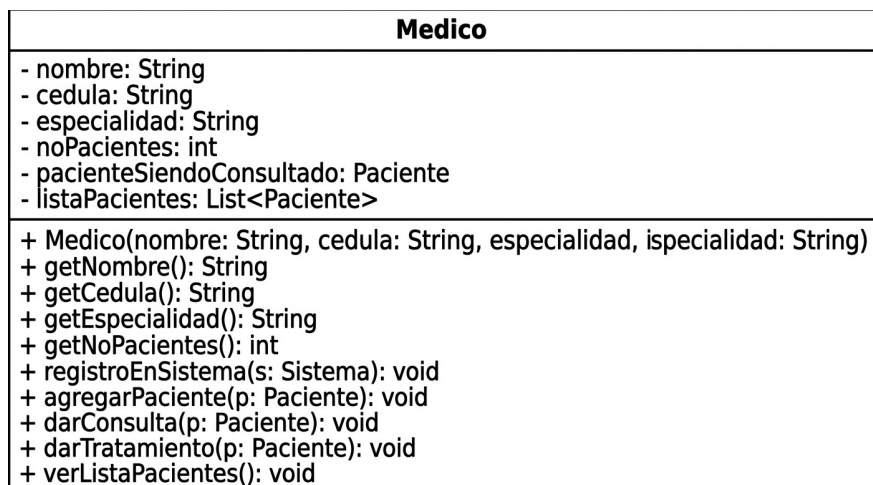


Figura 2: Diagrama de clase UML para Médico.

Enfermero
- nombre: String - cedula: String - especialidad: String - noPacientes: int - listaPacientes: List<Paciente>
+ Enfermero(nombre: String, cedula: String, especialidad: String) + getNombre(): String + getCedula(): String + getEspecialidad(): String + getNoPacientes(): int + registroEnSistema(s: Sistema): void + agregarPaciente(p: Paciente): void + darTratamiento(p: Paciente): void + verListaPacientes(): void

Figura 3: Diagrama de clase UML para Enfermero.

Paciente
- nombre: String - especialidadAtencion: String - enConsulta: boolean - enTratamiento: boolean
+ Paciente(nombre: String, especialidadAtencion: String) + getNombre(): String + getEspecialidadAtencion(): String + estaEnConsulta(): boolean + estaEnTratamiento(): boolean + setEnConsulta(estado: boolean): void + setEnTratamiento(estado: boolean): void + registroEnSistema(s: Sistema): void + solicitarConsulta(): void + verConsulta(): void + verTratamiento(): void

Figura 4: Diagrama de clase UML para Paciente.

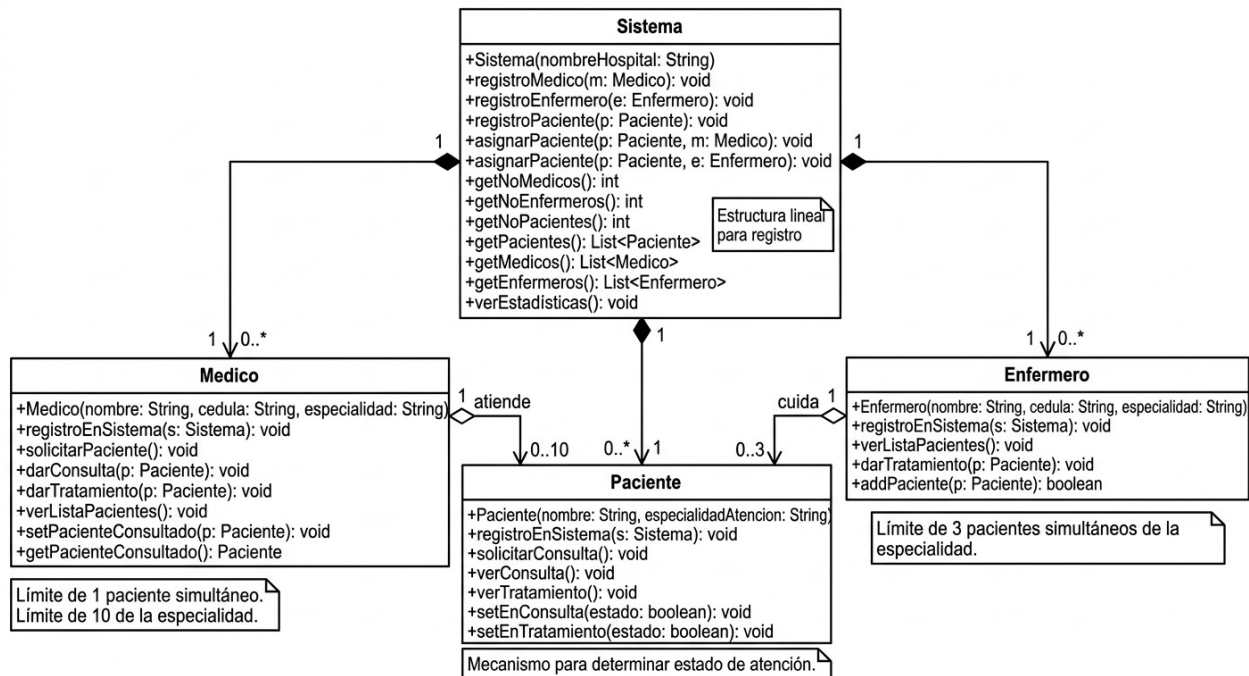


Figura 5: Diagrama de clases UML general del Sistema Hospitalario.

Prueba 1: Como prueba de validación de entrada se simuló un ingreso en blanco al presionar la tecla Enter sin escribir el nombre del hospital, verificando que el ciclo **do-while** atrapara el error y obligara al usuario a reingresar el dato.

Prueba 2: Se verificó la instanciación de objetos de la especialidad **"Cardiología"** y su registro, comprobando que tanto el método interno de los objetos (**registroEnSistema**) como el directo del controlador (**registroMedico**) incrementaran los contadores del sistema correctamente.

Prueba 3: Para la asignación de pacientes, se ingresaron objetos a la función sobrecargada **asignarPaciente**, validando que el sistema corroborara la coincidencia de especialidades y no permitiera exceder el límite de 1 paciente a la vez por médico (máximo 10) y 3 por enfermero.

Prueba 4: Se probó el mecanismo de estados clínicos evaluando las banderas booleanas de los pacientes. Se verificó el cambio de estado de **inactivo**.^a **"en consulta"**, comprobando que el sistema bloqueara la acción de dar tratamiento si el paciente seguía en la consulta previa.

Prueba 5: Para validar el requerimiento de ordenamiento, se invocó el método **verListaPacientes()**, comprobando que el algoritmo iterara sobre la estructura lineal y mostrara los nombres y especialidades organizados en formato alfabético descendente.

4. Resultados

Se muestra la advertencia de error en consola al simular el ingreso de un nombre de hospital vacío.

```
PS C:\Users\o-ale\OneDrive\Documentos\P00\Proyecto> java Main  
  
Ingresa un nombre para el Hospital:  
ERROR: no se ingresó nada, intenta de nuevo  
  
Ingresa un nombre para el Hospital: 
```

Figura 6: Prueba de error de cadena vacía.

Se muestra la verificación de los contadores y los registros del personal y pacientes ingresados en el sistema.

```
Ingresa un nombre para el Hospital: Angeles  
  
MÉTODOS DE REGISTRO  
  
GETTERS DE CONSULTA DE DATOS DEL SISTEMA  
  
Número de Médicos: 2  
Médicos: [Medico@5b6f7412, Medico@27973e9b]  
Número de Enfermeros: 2  
Enfermeros: [Enfermero@7530d0a, Enfermero@27bc2616]  
Número de Pacientes: 2  
Pacientes: [Paciente@3941a79c, Paciente@506e1b77]
```

Figura 7: Contadores y registros del hospital.

Se muestra la asignación exitosa de pacientes validando la coincidencia de especialidades y los límites de capacidad operativa.

```
INFORMACIÓN PACIENTES
Paciente solicita consulta.
Nombre del paciente: Juan | Especialidad requerida del paciente: Cardiología
Paciente solicita consulta.
Nombre del paciente: Ana | Especialidad requerida del paciente: Cardiología

ASIGNACIÓN MÉDICOS/ENFERMEROS A PACIENTES
Paciente asignado al enfermero de manera exitosa.
Paciente asignado al enfermero de manera exitosa.
Paciente asignado al médico de manera exitosa.
Paciente asignado al médico de manera exitosa.

MÉTODOS Y GETTER DE MÉDICOS/ENFERMEROS
Nombre de Médico 1: Andres
Cedula de Médico 1: 12458798
Especialidad de Médico 1: Cardiología
Número de pacientes del Médico 1: 1
Médico solicita un paciente.
Pacientes del Dr. Andres:
Juan (Cardiología).
Nombre de Enfermero 1: Pablo
Cedula de Enfermero 1: 9956789
Especialidad de Enfermero 1: Cardiología
Número de pacientes del Enfermero 1: 1
Pacientes del Dr. Andres:
Juan (Cardiología).
```

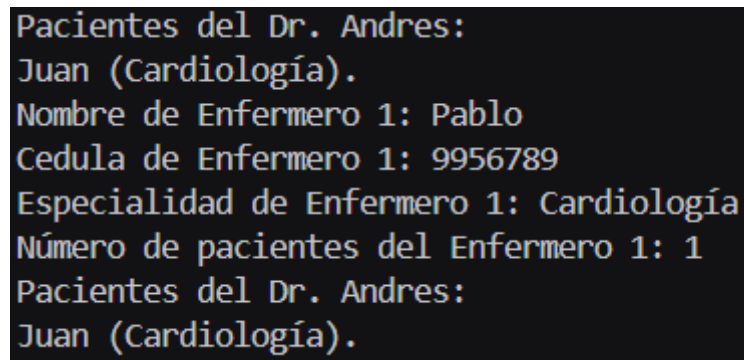
Figura 8: Asignación de pacientes.

Se muestra la transición lógica y bloqueo de los estados clínicos del paciente (en consulta y en tratamiento).

```
FLUJO CONSULTA Y TRATAMIENTO
El Dr. Andres está consultando a Juan.
El paciente está en consulta.
El Dr. Andres está dando tratamiento a Juan.
El paciente Ana no se encuentra en la lista del enfermero Pablo.
PS C:\Users\o-ale\OneDrive\Documentos\P00\Proyecto>
```

Figura 9: Control de flujo clínico.

Se muestra la impresión en terminal del listado de pacientes en estricto orden alfabético descendente.



```
Pacientes del Dr. Andres:  
Juan (Cardiología).  
Nombre de Enfermero 1: Pablo  
Cedula de Enfermero 1: 9956789  
Especialidad de Enfermero 1: Cardiología  
Número de pacientes del Enfermero 1: 1  
Pacientes del Dr. Andres:  
Juan (Cardiología).
```

Figura 10: Ordenamiento alfabético descendente.

5. Conclusiones

El desarrollo de esta primera etapa del proyecto permitió aplicar de manera integral los pilares de la programación orientada a objetos en un escenario práctico. Al abstraer el funcionamiento de un hospital en las clases **Sistema**, **Médico**, **Enfermero** y **Paciente** se evidenció la importancia del encapsulamiento para proteger la integridad de la información clínica y limitar el acceso mediante métodos *getters* y *setters*.

La implementación de la sobrecarga de métodos demostró ser una herramienta eficiente para simplificar la arquitectura del código, logrando que la acción de `asignarPaciente` se adaptara dinámicamente dependiendo de si el receptor era un médico o un enfermero. La adopción de estructuras de datos lineales dinámicas (como `ArrayList`) resultó fundamental para superar las limitantes de los arreglos estáticos, facilitando tanto el registro continuo como el ordenamiento alfabético de los usuarios.

Referencias

- [1] Introducción a la programación con Java (1.a ed.). (2009). Mc Graw Hill.
- [2] Olguín, J. A. S. G. R. A. S. (s. f.). Abstracción y encapsulamiento. Unidades de Apoyo Para el Aprendizaje - CUAED - UNAM. https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3076/mod_resource/content/1/UAPA-Abstraccion-Encapsulamiento/index.html
- [3] Programador Certificado Java 2. (2.a ed.). (1992). Alfaomega Grupo Editor.
- [4] ¿Qué es una clase en Java? | Netec. (s. f.). Netec. <https://www.netec.com/que-es-una-clase-en-java>
- [5] Sandoval, E. (2023, 21 septiembre). ¿Qué es un objeto en programación? Ebac. <https://ebac.mx/blog/objeto-en-programacion>
- [6] Sobrecarga de metodos y de constructores. (s.f.). http://profesores.fi-b.unam.mx/carlos/java/java_basico4_6.html
- [7] Team, D., Team, D., Team, D. (2026, 28 mayo). Linear Data Structures in Java – Array, Linked list, Stacks, and Queues. DataFlair. <https://data-flair.training/blogs/linear-data-structures-in-java/>